부록 · Docker 로 DB 설치

Docker MariaDB 설치 교본

이 문서에서 고친 곳

- 백업 명령을 고쳤습니다. -p 로 비밀번호를 묻게 하면 docker exec -i 에서는 입력을 제대로 받지 못하고, 받지 못하면 Access denied 와 함께 0바이트 백업 파일만 남습니다(실측). 컨테이너에 이미 있는 환경변수(compose 의 MARIADB_USER·MARIADB_PASSWORD)로 넘기면 비밀번호가 명령 기록에도 남지 않습니다.

Rocky Linux + Docker Compose · 초보자용

목표: MariaDB를 Docker로 설치하고, 설정파일 분리, 데이터 영구저장, devuser 관리자 계정, 외부 3306 접속, DBeaver 연동까지 한 번에 구성함.

1. 최종 구성

최종 디렉터리 구조는 다음과 같이 구성함.

```text
/home/devuser/docker/mariadb/
├── compose.yml       ← Docker Compose 설정
├── .env              ← DB명, 사용자, 비밀번호, 포트
├── config/
│   └── my.cnf        ← MariaDB 설정
├── init/
│   └── 01-init.sql   ← 최초 초기화 SQL
└── data/             ← 실제 DB 데이터 영구 저장
```

가장 중요한 폴더는 ~/docker/mariadb/data 임. 컨테이너를 삭제하거나 재생성해도 이 폴더를 삭제하지 않으면 데이터가 유지됨.

2. Docker 설치 여부 확인

```bash
docker --version
docker compose version
sudo systemctl status docker
```

Docker가 실행 중이 아니면 다음 명령으로 시작하고 부팅 시 자동 실행되도록 설정함.

```bash
sudo systemctl enable --now docker
```

3. 기존 MariaDB와 3306 포트 확인

```bash
rpm -qa | grep -Ei 'mariadb|MariaDB'
sudo ss -lntp | grep 3306
```

기존 MariaDB를 삭제할 계획이라면 필요한 데이터는 반드시 먼저 백업함. 3306 포트가 비어 있어야 Docker MariaDB가 기본 포트를 사용할 수 있음.

4. 작업 디렉터리 생성

```bash
mkdir -p ~/docker/mariadb/{data,config,init}
cd ~/docker/mariadb
pwd
ls -la
```

5. .env 환경설정 파일

```bash
nano .env
```

```env
MARIADB_ROOT_PASSWORD=본인이정한root비밀번호
MARIADB_DATABASE=wizdb
MARIADB_USER=devuser
MARIADB_PASSWORD=본인이정한devuser비밀번호
MARIADB_PORT=3306
```

```bash
chmod 600 .env
```

비밀번호는 실제 운영 시 충분히 길고 추측하기 어려운 값으로 설정함.

6. MariaDB 설정파일 my.cnf

```bash
nano config/my.cnf
```

```ini
[mariadb]
bind-address=0.0.0.0
port=3306
character-set-server=utf8mb4
collation-server=utf8mb4_unicode_ci
default-time-zone=+09:00
max_connections=200
slow_query_log=1
long_query_time=2

[client]
default-character-set=utf8mb4
```

bind-address=0.0.0.0 은 컨테이너가 외부 연결을 받을 수 있게 하는 설정임. 실제 접근 범위는 방화벽에서 내부망 등으로 제한하는 것이 안전함.

7. devuser 전체 권한 초기화

```bash
nano init/01-init.sql
```

```sql
GRANT ALL PRIVILEGES ON *.*
TO 'devuser'@'%'
WITH GRANT OPTION;

FLUSH PRIVILEGES;
```

이 SQL은 빈 data 디렉터리에서 최초 초기화할 때 실행됨. devuser 계정에 모든 DB에 대한 전체 권한과 권한 부여 권한을 설정함.

8. compose.yml 전체 설정

```bash
nano compose.yml
```

```yaml
services:
  mariadb:
    image: mariadb:11.8
    container_name: wiz-mariadb
    restart: unless-stopped

    env_file:
      - .env

    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
      MARIADB_DATABASE: ${MARIADB_DATABASE}
      MARIADB_USER: ${MARIADB_USER}
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}

    ports:
      - "${MARIADB_PORT}:3306"

    volumes:
      - ./data:/var/lib/mysql:Z
      - ./config/my.cnf:/etc/mysql/conf.d/99-wiz.cnf:ro,Z
      - ./init:/docker-entrypoint-initdb.d:ro,Z

    healthcheck:
      test: ["CMD-SHELL", "mariadb-admin ping -h localhost -u root -p$$MARIADB_ROOT_PASSWORD --silent"]
      interval: 10s
      timeout: 5s
      retries: 10

    networks:
      - db-network

networks:
  db-network:
    driver: bridge
```

Rocky Linux의 SELinux 환경을 고려하여 bind mount에 :Z 옵션을 사용함. YAML은 TAB이 아니라 공백으로 들여쓰기해야 함.

9. Compose 문법 검사 및 실행

```bash
docker compose config
docker compose up -d
docker ps
```

```bash
docker compose config 에 오류가 없을 때만 실행함. 최초 실행 시 MariaDB 이미지를 다운로드하므로 시간이 조금 걸릴 수 있음.
```

10. 로그 및 포트 확인

```bash
docker logs wiz-mariadb
sudo ss -lntp | grep 3306
```

정상이라면 0.0.0.0:3306 또는 [::]:3306 에 docker-proxy가 LISTEN 중인 것을 확인할 수 있음.

11. MariaDB 접속

devuser 계정으로 wizdb에 바로 접속하는 가장 중요한 명령임.

```bash
docker exec -it wiz-mariadb mariadb -u devuser -p wizdb
```

접속 후 기본 명령:

```sql
SHOW DATABASES;
SELECT DATABASE();
SHOW TABLES;
SELECT USER(), CURRENT_USER();
SHOW GRANTS FOR 'devuser'@'%';
exit;
```

12. users 예제 테이블

```sql
USE wizdb;

CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(20),
    role VARCHAR(20) DEFAULT 'USER',
    enabled BOOLEAN DEFAULT TRUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP
);
```

```sql
INSERT INTO users
(username, password, name, email, phone, role)
VALUES
('hong', '1234', '홍길동', 'hong@test.com', '010-3333-4444', 'USER');
```

```sql
SHOW TABLES;
DESC users;
SELECT * FROM users;
SELECT * FROM users\G
```

MariaDB에서 SELECT 결과를 세로로 보고 싶으면 쿼리 끝에 \G 를 사용함. 실제 로그인 시스템에서는 평문 비밀번호를 저장하지 않고 BCrypt 등의 해시 방식을 사용함.

13. Rocky Linux 방화벽 - 내부망만 허용

3306을 인터넷 전체에 개방하기보다 내부망 192.168.0.0/24에서만 허용하는 구성을 권장함.

```bash
sudo firewall-cmd --permanent --remove-port=3306/tcp

sudo firewall-cmd --permanent \
  --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="3306" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

14. Windows에서 포트 테스트

Rocky Linux 서버 내부 IP가 SERVER_IP인 현재 환경에서는 Windows PowerShell에서 다음처럼 확인함.

```powershell
Test-NetConnection SERVER_IP -Port 3306
```

TcpTestSucceeded : True 가 나오면 Windows에서 Rocky Linux의 Docker MariaDB 3306 포트까지 네트워크 연결이 성공한 것임.

15. DBeaver 연결

항목	값
DB 종류	MariaDB
Host	SERVER_IP
Port	3306
Database	wizdb
Username	devuser
Password	.env에서 설정한 비밀번호

```text
jdbc:mariadb://SERVER_IP:3306/wizdb
```

16. 자주 사용하는 Docker 관리 명령

목적	명령어
Compose 설정 검사	docker compose config
MariaDB 시작	docker compose up -d
실행 상태	docker ps
중지	docker compose stop
재시작	docker compose restart

목적	명령어
컨테이너 제거	docker compose down
로그	docker logs wiz-mariadb
실시간 로그	docker logs -f wiz-mariadb
DB 접속	docker exec -it wiz-mariadb mariadb -u devuser -p wizdb

> restart: unless-stopped 설정 때문에 Docker 서비스가 정상적으로 시작되면 서버 재부팅 후 컨테이너도 자동 재시작 yaml
> 됨.

17. 데이터 영구저장 원리

```text
Rocky Linux
/home/devuser/docker/mariadb/data
              │
              │ Bind Mount
              ▼
Docker 컨테이너
/var/lib/mysql
```

> docker compose down 으로 컨테이너를 제거해도 호스트의 data 폴더는 유지됨. 다시 docker compose up -d 하면 bash
> 기존 데이터 디렉터리를 사용함.

주의:

> ~/docker/mariadb/data 디렉터리는 DB 원본 데이터이므로 함부로 삭제하면 안 됨. text

18. 백업 기본

간단한 수동 백업은 mariadb-dump를 이용함. 비밀번호를 명령행에 직접 적는 방식은 shell history에 남을 수 있으므로 운영 자동화 시 별도 자격증명 파일 사용을 권장함.

```bash
docker exec wiz-mariadb sh -c \
  'exec mariadb-dump -u"$MARIADB_USER" -p"$MARIADB_PASSWORD" "$MARIADB_DATABASE"' > ~/wizdb_ba
ckup.sql
```

19. 초보자가 기억할 핵심 명령 7개

```bash
cd ~/docker/mariadb
docker compose config
docker compose up -d
docker ps
docker logs wiz-mariadb
docker exec -it wiz-mariadb mariadb -u devuser -p wizdb
docker compose down
```

20. 최종 점검표

점검 항목	정상 기준
Docker Compose	docker compose config 오류 없음
컨테이너	wiz-mariadb 실행 중
영구저장	~/docker/mariadb/data 사용
DB	wizdb 존재
사용자	devuser 존재
권한	devuser 전체 권한
외부 포트	3306 LISTEN
방화벽	192.168.0.0/24 → 3306 허용
Windows 테스트	TcpTestSucceeded : True
GUI 관리	DBeaver에서 연결 가능

끝 - Rocky Linux Docker MariaDB 초보자 설치·운영 교본

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · Docker 로 DB 설치

Docker PostgreSQL 설치 교본

이 문서에서 고친 곳

- \gx 뒤의 세미콜론을 뺐습니다. \gx 자체가 끝 표시라 붙이면 「invalid command \gx;」 오류가 납니다(실측).
- pg_dump 를 받을 때 docker exec -t 를 뺐습니다. -t(터미널)를 붙이면 줄 끝이 CRLF 로 바뀌어 덤프 파일이 오염됩니다 (실측: CR 41개 → 0개).

Rocky Linux + Docker Compose · 초보자용

목표: PostgreSQL을 Docker Compose로 설치하고 설정파일 분리, 데이터 영구저장, devuser 계정, 외부 5432 접속, DBeaver 연동까지 구성함.

1. 최종 구성

```text
/home/devuser/docker/postgresql/
├── compose.yml
├── .env
├── .psqlrc
├── config/
│   ├── postgresql.conf
│   └── pg_hba.conf
├── init/
│   └── 01-init.sql
└── data/                ← 실제 PostgreSQL 데이터 영구 저장
```

가장 중요한 디렉터리는 ~/docker/postgresql/data 임. 컨테이너를 삭제하거나 재생성해도 이 폴더를 삭제하지 않으면 DB 데이터가 유지됨.

2. Docker 설치 여부 확인

```bash
docker --version
docker compose version
sudo systemctl status docker
```

```bash
sudo systemctl enable --now docker
```

3. 기존 PostgreSQL 및 5432 포트 확인

```bash
rpm -qa | grep -Ei 'postgresql|postgres'
sudo ss -lntp | grep 5432
```

직접 설치된 PostgreSQL을 제거하려면 필요한 DB를 먼저 백업해야 함. 5432가 비어 있어야 Docker PostgreSQL이 기본 포트를 사용할 수 있음.

4. 디렉터리 생성

```bash
mkdir -p ~/docker/postgresql/{data,config,init}
cd ~/docker/postgresql
pwd
ls -la
```

5. .env 환경설정

```bash
nano .env
```

```env
POSTGRES_DB=wizdb
POSTGRES_USER=devuser
POSTGRES_PASSWORD=본인이정한강력한비밀번호
POSTGRES_PORT=5432
```

```bash
chmod 600 .env
```

POSTGRES_USER와 POSTGRES_DB는 빈 data 디렉터리에서 최초 초기화할 때 생성됨.

6. postgresql.conf 만들기

```bash
nano config/postgresql.conf
```

```text
listen_addresses = '*'
port = 5432

max_connections = 100
shared_buffers = 256MB
work_mem = 4MB
maintenance_work_mem = 64MB

logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d.log'

timezone = 'Asia/Seoul'
log_timezone = 'Asia/Seoul'

password_encryption = 'scram-sha-256'
```

listen_addresses='*' 는 모든 네트워크 인터페이스에서 연결을 받을 수 있게 함. 실제 허용 IP는 pg_hba.conf와 방화벽에서 제한하는 것이 안전함.

7. pg_hba.conf 만들기

```bash
nano config/pg_hba.conf
```

```
local   all       all                     trust                                   text
host    all       all   127.0.0.1/32        scram-sha-256
host    all       all   ::1/128             scram-sha-256

# 내부망에서만 외부 접속 허용
host    all       all   192.168.0.0/24      scram-sha-256
```

0.0.0.0/0 으로 허용할 수도 있지만 인터넷 전체를 인증 대상으로 만들 수 있으므로 내부망 사용 시 192.168.0.0/24처럼 제한하는 것을 권장함.

8. 최초 초기화 SQL

```
nano init/01-init.sql                                                           bash
```

```
CREATE SCHEMA IF NOT EXISTS wiz;                                                sql

CREATE TABLE IF NOT EXISTS wiz.member (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(200),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

docker-entrypoint-initdb.d 아래의 SQL은 data 디렉터리가 비어 있는 최초 초기화 때만 자동 실행됨.

9. psql 편의 설정

```
nano .psqlrc                                                                    bash
```

```
\x auto                                                                         sql
\pset null '(NULL)'
\timing on
```

\x auto는 결과가 화면 폭보다 길 때 자동으로 세로 확장 출력함. 항상 세로는 \x on, 항상 가로는 \x off 임.

10. compose.yml 전체 설정

```
nano compose.yml                                                                bash
```

```yaml
services:
  postgres:
    image: postgres:17
    container_name: wiz-postgresql
    restart: unless-stopped

    env_file:
      - .env

    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

    ports:
      - "${POSTGRES_PORT}:5432"

    volumes:
      - ./data:/var/lib/postgresql/data:Z
      - ./config/postgresql.conf:/etc/postgresql/postgresql.conf:ro,Z
      - ./config/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro,Z
      - ./init:/docker-entrypoint-initdb.d:ro,Z
      - ./.psqlrc:/root/.psqlrc:ro,Z

    command:
      - postgres
      - -c
      - config_file=/etc/postgresql/postgresql.conf
      - -c
      - hba_file=/etc/postgresql/pg_hba.conf

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

    networks:
      - db-network

networks:
  db-network:
    driver: bridge
```

Rocky Linux SELinux 환경을 고려하여 bind mount에 :Z 옵션을 사용함. YAML은 TAB이 아니라 공백으로 들여쓰기해야 함.

11. Compose 문법 검사 및 시작

```bash
docker compose config
docker compose up -d
docker ps
```

```bash
docker compose config에서 YAML 오류가 없는 것을 확인한 뒤 up -d를 실행함.
```

12. 로그 및 5432 확인

```bash
docker logs wiz-postgresql
sudo ss -lntp | grep 5432
```

정상이라면 0.0.0.0:5432 또는 [::]:5432에 docker-proxy가 LISTEN 중인 것을 확인할 수 있음.

13. devuser으로 PostgreSQL 접속

```bash
docker exec -it wiz-postgresql psql -U devuser -d wizdb
```

정상 접속되면 wizdb=# 또는 wizdb=> 프롬프트가 표시됨.

```sql
SELECT current_user, current_database();
\l
\du
\dn
\dt
\q
```

14. users 예제 테이블

```sql
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(20),
    role VARCHAR(20) DEFAULT 'USER',
    enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
INSERT INTO users
(username, password, name, email, phone, role)
VALUES
('hong', '1234', '홍길동', 'hong@test.com', '010-3333-4444', 'USER');

SELECT * FROM users;
SELECT * FROM users \gx
```

실제 로그인 시스템에서는 평문 비밀번호를 저장하지 않고 BCrypt 등 안전한 해시 방식을 사용해야 함.

15. 테이블과 구조 확인

```sql
\dt
\d users
SELECT * FROM users;
```

psql에서 \x on은 계속 세로 출력, \x off는 가로 출력, \x auto는 자동 선택, SELECT ... \gx는 해당 조회만 세로 출력함.

16. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 5432 접속을 허용하는 예임.

```bash
sudo firewall-cmd --permanent \
  --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="5432" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

17. Windows에서 포트 테스트

현재 Rocky Linux 서버 내부 IP가 SERVER_IP인 환경 기준:

```powershell
Test-NetConnection SERVER_IP -Port 5432
```

TcpTestSucceeded : True가 나오면 Windows PC에서 Docker PostgreSQL 포트까지 네트워크 연결이 성공한 것임.

18. DBeaver 연결

항목	값
DB 종류	PostgreSQL
Host	SERVER_IP
Port	5432
Database	wizdb
Username	devuser
Password	.env에 설정한 비밀번호

```text
jdbc:postgresql://SERVER_IP:5432/wizdb
```

19. devuser 권한 확인 및 관리

```sql
\du
SELECT current_user;
```

```env
POSTGRES_USER=devuser으로 최초 초기화하면 공식 PostgreSQL 이미지에서 해당 사용자는 초기 관리자 역할로 생성됨.
일반 애플리케이션 운영에서는 별도의 최소권한 계정을 추가하는 것이 더 안전함.
```

```sql
CREATE USER appuser WITH PASSWORD '강력한비밀번호';
GRANT CONNECT ON DATABASE wizdb TO appuser;
```

20. 자주 쓰는 Docker 명령

목적	명령
설정 검사	docker compose config

목적	명령
시작	docker compose up -d
상태	docker ps
중지	docker compose stop
재시작	docker compose restart
컨테이너 제거	docker compose down
로그	docker logs wiz-postgresql
실시간 로그	docker logs -f wiz-postgresql
DB 접속	docker exec -it wiz-postgresql psql -U devuser -d wizdb

21. 데이터 영구저장 원리

```text
Rocky Linux
/home/devuser/docker/postgresql/data
                 │
                 │ Bind Mount
                 ▼
Docker PostgreSQL
/var/lib/postgresql/data
```

```bash
docker compose down으로 컨테이너를 제거해도 호스트의 data 폴더는 남음. 다시 docker compose up -d 하면 기존 DB를 사용함.
```

주의:

```text
~/docker/postgresql/data 디렉터리는 실제 DB 원본이므로 함부로 삭제하면 안 됨.
```

22. 백업과 복구 기초

특정 DB를 SQL 파일로 백업하는 기본 예임.

```bash
docker exec wiz-postgresql \
  pg_dump -U devuser -d wizdb > ~/wizdb_postgresql_backup.sql
```

복구 전에는 대상 DB와 권한을 확인해야 하며, 운영 환경에서는 정기 백업과 별도 저장소 보관을 권장함.

23. 초보자가 기억할 핵심 명령 7개

```bash
cd ~/docker/postgresql
docker compose config
docker compose up -d
docker ps
docker logs wiz-postgresql
docker exec -it wiz-postgresql psql -U devuser -d wizdb
docker compose down
```

24. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-postgresql 실행 중
영구저장	~/docker/postgresql/data 사용
DB	wizdb 존재
사용자	devuser 접속 가능
외부 포트	5432 LISTEN
pg_hba.conf	192.168.0.0/24 허용
방화벽	192.168.0.0/24 → 5432 허용
Windows 테스트	TcpTestSucceeded : True
GUI	DBeaver 연결 가능

부록 · Docker 로 DB 설치

Docker MongoDB 설치 교본

Rocky Linux + Docker Compose · MongoDB Community · 초보자용

목표: MongoDB Community 공식 컨테이너 이미지를 Docker Compose로 설치하고, 설정파일 분리, 데이터 영구저장, 관리자 계정과 devuser 애플리케이션 계정, 27017 내부망 접속, MongoDB Compass/DBeaver 연결까지 구성함.

이 문서에서 고친 곳

- 백업 단계에 docker cp 를 더했습니다. 원문은 ~/mongodb-backup 폴더를 만들어 놓고 실제 덤프는 컨테이너 안 /tmp/backup 에 남겨, 컨테이너를 지우면 백업도 함께 사라졌습니다.

1. MongoDB와 관계형 DB의 차이

구분	MariaDB/PostgreSQL/Oracle	MongoDB
구조	DB → Table → Row	DB → Collection → Document
데이터 형식	행/열	BSON 문서
기본 포트	3306/5432/1521	27017
대표 CLI	mariadb/psql/sqlplus	mongosh

MongoDB에서는 테이블 대신 collection, 행 대신 document라는 용어를 사용함.

2. 최종 디렉터리 구조

```text
/home/devuser/docker/mongodb/
├── compose.yml
├── .env
├── config/
│   └── mongod.conf
├── init/
│   └── 01-init.js
└── data/                  ← 실제 MongoDB 데이터 영구 저장
```

MongoDB 공식 Community 이미지의 데이터 경로는 /data/db 이므로 이를 ~/docker/mongodb/data에 bind mount하여 영구 저장함.

3. Docker 확인

```bash
docker --version
docker compose version
sudo systemctl status docker
```

```bash
sudo systemctl enable --now docker
```

4. 기존 MongoDB와 27017 확인

```bash
rpm -qa | grep -Ei 'mongodb|mongosh'
sudo ss -lntp | grep 27017
docker ps -a
```

기존 MongoDB가 있으면 필요한 데이터를 먼저 백업한 뒤 포트 충돌 여부를 확인함.

5. 디렉터리 생성

```bash
mkdir -p ~/docker/mongodb/{data,config,init}
cd ~/docker/mongodb
pwd
ls -la
```

6. .env 환경설정

```bash
nano .env
```

```env
MONGO_ROOT_USERNAME=mongoadmin
MONGO_ROOT_PASSWORD=본인이정한강력한관리자비밀번호
MONGO_DATABASE=wizdb
MONGO_APP_USERNAME=devuser
MONGO_APP_PASSWORD=본인이정한devuser비밀번호
MONGO_PORT=27017
```

```bash
chmod 600 .env
```

관리자 계정과 애플리케이션용 devuser 계정을 분리하는 구조임. 비밀번호는 충분히 강하게 설정함.

7. mongod.conf 만들기

```bash
nano config/mongod.conf
```

```yaml
storage:
  dbPath: /data/db

net:
  port: 27017
  bindIpAll: true

security:
  authorization: enabled
```

bindIpAll은 모든 인터페이스에서 listener가 대기하도록 함. 실제 접근 범위는 Rocky Linux 방화벽으로 192.168.0.0/24 등 필요한 네트워크에 제한함.

8. devuser 초기화 스크립트

```bash
nano init/01-init.js
```

```javascript
db = db.getSiblingDB('wizdb');

db.createUser({
  user: 'devuser',
  pwd: '본인이정한devuser비밀번호',
  roles: [
    { role: 'readWrite', db: 'wizdb' }
  ]
});

db.createCollection('users');

db.users.insertOne({
  username: 'hong',
  name: '홍길동',
  email: 'hong@test.com',
  phone: '010-3333-4444',
  role: 'USER',
  enabled: true,
  createdAt: new Date()
});
```

주의: 위 초기화 파일의 devuser 비밀번호는 .env의 MONGO_APP_PASSWORD와 동일하게 직접 맞춰야 함. docker-entrypoint-initdb.d 스크립트는 data가 비어 있는 최초 초기화 때만 실행됨.

9. compose.yml 만들기

```bash
nano compose.yml
```

```javascript
services:
  mongodb:
    image: mongodb/mongodb-community-server:8.0-ubi8
    container_name: wiz-mongodb
    restart: unless-stopped

    environment:
      MONGODB_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGODB_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}

    ports:
      - "${MONGO_PORT}:27017"

    volumes:
      - ./data:/data/db:Z
      - ./config/mongod.conf:/etc/mongod.conf:ro,Z
      - ./init:/docker-entrypoint-initdb.d:ro,Z

    command: ["mongod", "--config", "/etc/mongod.conf"]

    healthcheck:
      test: ["CMD-SHELL", "mongosh --quiet --eval 'db.runCommand({ ping: 1 }).ok' mongodb://
$$MONGODB_INITDB_ROOT_USERNAME:$$MONGODB_INITDB_ROOT_PASSWORD@localhost:27017/admin?authSource
=admin | grep 1"]
      interval: 15s
      timeout: 10s
      retries: 10
      start_period: 30s
```

Rocky Linux의 SELinux 환경을 고려해 bind mount에 :Z를 사용함. MongoDB 공식 문서는 Community Docker 이미지를 제공하며 기본 데이터 경로로 /data/db를 사용함.

10. Compose 문법 검사

```bash
docker compose config
```

오류가 없으면 다음 단계로 진행함. YAML에서는 TAB 대신 공백을 사용함.

11. 이미지 다운로드 및 실행

```bash
docker compose pull
docker compose up -d
docker ps
```

처음에는 이미지를 내려받고 MongoDB 데이터 디렉터리를 초기화하므로 잠시 기다림.

12. 로그와 포트 확인

```bash
docker logs wiz-mongodb
sudo ss -lntp | grep 27017
```

정상이라면 27017 포트에서 docker-proxy가 LISTEN 중임.

13. 관리자 계정으로 mongosh 접속

```bash
docker exec -it wiz-mongodb mongosh  -u mongoadmin  -p  --authenticationDatabase admin
```

비밀번호를 묻는 경우 .env의 MONGO_ROOT_PASSWORD를 입력함.

```javascript
show dbs
use admin
db.getUsers()
exit
```

14. devuser으로 wizdb 접속

```bash
docker exec -it wiz-mongodb mongosh  "mongodb://devuser@localhost:27017/wizdb?authSource=
wizdb"  --password
```

비밀번호를 입력하면 wizdb에 접속함.

15. MongoDB 기본 명령

```javascript
show dbs
use wizdb
show collections
db.users.find()
db.users.find().pretty()
db.users.countDocuments({})
db.users.findOne({ username: "hong" })
```

16. users 컬렉션 직접 만들기

```javascript
use wizdb

db.createCollection("users")

db.users.insertOne({
  username: "kim",
  name: "김철수",
  email: "kim@test.com",
  phone: "010-1111-2222",
  role: "ADMIN",
  enabled: true,
  createdAt: new Date()
})
```

17. CRUD 연습

```text
// CREATE
db.users.insertOne({
  username: "lee",
  name: "이순신",
  email: "lee@test.com",
  role: "USER",
  enabled: true
})

// READ
db.users.find()
db.users.findOne({ username: "lee" })

// UPDATE
db.users.updateOne(
  { username: "lee" },
  { $set: { email: "lee2@test.com" } }
)

// DELETE
db.users.deleteOne({ username: "lee" })
```

18. devuser 권한 확인

```javascript
use wizdb
db.getUser("devuser")
```

현재 예제에서는 devuser에 wizdb의 readWrite 권한만 부여함. 애플리케이션 계정에는 root 권한보다 필요한 DB의 readWrite 정도를 부여하는 것이 안전함.

19. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 MongoDB 27017 접속을 허용하는 예임.

```bash
sudo firewall-cmd --permanent --remove-port=27017/tcp

sudo firewall-cmd --permanent   --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="27017" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

20. Windows에서 포트 테스트

현재 서버 내부 IP SERVER_IP 기준:

```powershell
Test-NetConnection SERVER_IP -Port 27017
```

TcpTestSucceeded : True이면 Windows에서 Docker MongoDB 포트까지 연결됨.

21. MongoDB Compass 연결

MongoDB GUI로는 MongoDB Compass를 사용하면 편리함. devuser은 wizdb에 인증되는 계정이므로 authSource=wizdb를 지정함.

```text
mongodb://devuser:비밀번호@SERVER_IP:27017/wizdb?authSource=wizdb
```

관리자 접속은 authSource=admin을 사용함.

```text
mongodb://mongoadmin:관리자비밀번호@SERVER_IP:27017/admin?authSource=admin
```

22. DBeaver 연결

DBeaver의 MongoDB 지원 범위는 사용하는 에디션/드라이버에 따라 달라질 수 있음. MongoDB 전용 관리에는 Compass가 가장 직관적이며, DBeaver에서 MongoDB 연결 옵션이 제공되는 환경이라면 같은 Host/Port/사용자/인증 DB 정보를 사용함.

항목	값
Host	SERVER_IP
Port	27017
Database	wizdb
Username	devuser
Authentication DB	wizdb

23. 데이터 영구저장 원리

```text
Rocky Linux
/home/devuser/docker/mongodb/data
              |
              | Bind Mount
              ▼
MongoDB Container
/data/db
```

```bash
docker compose down을 실행해도 ~/docker/mongodb/data는 남음. 다시 docker compose up -d 하면 기존 데이터를 사용함.
```

주의:

```text
~/docker/mongodb/data는 실제 DB 원본이므로 함부로 삭제하지 않음.
```

24. 자주 사용하는 Docker 명령

목적	명령
설정 검사	docker compose config
이미지 다운로드	docker compose pull

목적	명령
시작	docker compose up -d
상태	docker ps
중지	docker compose stop
재시작	docker compose restart
컨테이너 제거	docker compose down
로그	docker logs wiz-mongodb
실시간 로그	docker logs -f wiz-mongodb
관리자 접속	docker exec -it wiz-mongodb mongosh -u mongoadmin -p --authenticationDatabase admin

25. 백업과 복구 기초

MongoDB의 논리 백업은 mongodump, 복구는 mongorestore를 사용함. 운영 데이터는 단순히 실행 중인 data 디렉터리를 복사하는 방식만 믿지 말고 공식 백업 도구 또는 적절한 스냅샷 전략을 사용함.

```bash
mkdir -p ~/mongodb-backup

docker exec wiz-mongodb  mongodump  --username mongoadmin  --password '관리자비밀번호'  --auth
enticationDatabase admin  --db wizdb  --out /tmp/backup

# 컨테이너 안의 백업을 밖(호스트)으로 꺼낸다 — 이게 없으면 컨테이너를 지울 때 백업도 사라진다
docker cp wiz-mongodb:/tmp/backup ~/mongodb-backup/
```

비밀번호를 명령행에 직접 쓰면 shell history 등에 남을 수 있으므로 실제 운영 자동화에서는 자격증명 노출을 피하는 방식을 사용함.

26. 초보자가 기억할 핵심 명령

```bash
cd ~/docker/mongodb
docker compose config
docker compose up -d
docker ps
docker logs wiz-mongodb
sudo ss -lntp | grep 27017
docker exec -it wiz-mongodb mongosh -u mongoadmin -p --authenticationDatabase admin
docker compose down
```

27. MariaDB·PostgreSQL·Oracle·MongoDB 비교

DB	포트	CLI	데이터 개념	영구저장 컨테이너 경로
MariaDB	3306	mariadb	Table/Row	/var/lib/mysql
PostgreSQL	5432	psql	Table/Row	/var/lib/postgresql/data
Oracle	1521	sqlplus	Table/Row	/opt/oracle/oradata
MongoDB	27017	mongosh	Collection/Document	/data/db

28. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-mongodb 실행 중
영구저장	~/docker/mongodb/data 사용
Listener	27017 LISTEN
관리자	mongoadmin 로그인 가능
DB	wizdb 사용 가능
앱 사용자	devuser 로그인 가능
권한	devuser → wizdb readWrite
방화벽	192.168.0.0/24 → 27017 허용
Windows	TcpTestSucceeded : True
GUI	MongoDB Compass 연결 가능

29. 보안상 꼭 기억할 점

MongoDB 27017 포트를 인터넷 전체에 그대로 공개하지 않는 것을 권장함. 인증을 활성화하고 방화벽으로 필요한 내부망 또는 애플리케이션 서버만 허용함. 관리자 계정과 애플리케이션 계정을 분리하고, 애플리케이션에는 최소 권한을 부여함.

부록 · Docker 로 DB 설치

Docker MongoDB8 초보자 실습 교본

Docker MongoDB 8 초보자 실습 교본

Rocky Linux + Docker + MongoDB 8 + mongosh 설치 · 실행 · 사용자 · 컬렉션 · CRUD · 검색 · 인덱스 · 100개 명령어

실습 항목	기준값
컨테이너 이름	wiz-mongodb
MongoDB 이미지	mongo:8
호스트 포트	27017
관리자 계정	root (admin DB)
실습 DB	wizdb
실습 사용자	devuser
실습 컬렉션	users

※ 비밀번호는 문서에 고정값으로 저장하지 말고 실제 서버에서 안전한 값으로 설정하는 것을 권장함.

1. MongoDB를 SQL DB와 비교해서 이해하기

관계형 DB	MongoDB	설명
Database	Database	데이터베이스
Table	Collection	문서(Document)를 담는 집합
Row	Document	한 건의 데이터
Column	Field	문서 내부의 항목
Primary Key	_id	MongoDB가 기본적으로 자동 생성
INSERT	insertOne / insertMany	데이터 입력
SELECT	find / findOne	데이터 조회
UPDATE	updateOne / updateMany	데이터 수정
DELETE	deleteOne / deleteMany	데이터 삭제

2. Docker MongoDB 기본 구성

아래 docker-compose.yml 예제는 MongoDB 데이터를 ./data에 영구 보관하고, 서버의 27017 포트를 컨테이너의 27017 포트에 연결함.

```yaml
services:
  mongodb:
    image: mongo:8
    container_name: wiz-mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: "안전한_관리자_비밀번호"
    volumes:
      - ./data:/data/db
```

```bash
cd ~/docker/mongodb
docker compose up -d
docker ps
docker logs --tail 30 wiz-mongodb
```

3. MongoDB 접속과 사용자 만들기

관리자(root)는 admin DB에 인증하고, 응용프로그램용 devuser 사용자는 wizdb에 readWrite 권한으로 생성하는 방식이 이해하기 쉬움.

```bash
docker exec -it wiz-mongodb mongosh -u root -p --authenticationDatabase admin
```

```javascript
use wizdb
db.createUser({
  user: "devuser",
  pwd: "여기에_안전한_비밀번호",
  roles: [
    { role: "readWrite", db: "wizdb" }
  ]
})
show users
```

```bash
docker exec -it wiz-mongodb mongosh -u devuser -p --authenticationDatabase wizdb wizdb
```

비밀번호 변경은 사용자가 만들어진 DB에서 실행함.

```javascript
use wizdb
db.changeUserPassword("devuser", "새로운_안전한_비밀번호")
```

4. users 컬렉션 만들기

```javascript
use wizdb
db.createCollection("users")
show collections
```

MongoDB에서는 컬렉션을 명시적으로 만들지 않고 insertOne()을 처음 실행해도 자동 생성될 수 있으나, 학습 단계에서는 createCollection()으로 직접 만드는 것이 구조를 이해하기 좋음.

5. CREATE - 데이터 입력

5.1 한 명 입력

```javascript
db.users.insertOne({
  user_id: 1,
  username: "hong",
  name: "홍길동",
  age: 30,
  gender: "남",
  phone: "010-1111-1111",
  email: "hong@test.com",
  address: "서울시 강남구",
  job: "개발자",
  salary: 5000000,
  active: true,
  reg_date: new Date()
})
```

5.2 여러 명 입력

```javascript
db.users.insertMany([
  { user_id: 2, username: "kim",  name: "김철수", age: 25, job: "디자이너", salary: 3500000 },
  { user_id: 3, username: "lee",  name: "이영희", age: 28, job: "회계사",   salary: 4200000 },
  { user_id: 4, username: "park", name: "박민수", age: 35, job: "교사",     salary: 4500000 },
  { user_id: 5, username: "kang", name: "강감찬", age: 40, job: "회사원",   salary: 4800000 }
])
```

6. READ - 데이터 조회

```javascript
db.users.find()
db.users.findOne({ username: "kim" })
db.users.find({ age: { $gte: 30 } })
db.users.find({ salary: { $gte: 4500000 } })
```

원하는 필드만 깔끔하게 출력하려면 Projection을 사용함. 1은 표시, 0은 제외를 의미함.

```javascript
db.users.find(
  {},
  { _id: 0, name: 1, age: 1, job: 1 }
)
```

```javascript
db.users.findOne(
  { name: "홍길동" },
  { _id: 0, name: 1, phone: 1, email: 1 }
)
```

7. UPDATE - 데이터 수정

```javascript
db.users.updateOne(
  { username: "kim" },
  { $set: { age: 26 } }
)
```

```javascript
db.users.updateOne(
  { username: "kim" },
  { $inc: { salary: 100000 } }
)
```

```javascript
db.users.updateMany(
  { age: { $gte: 30 } },
  { $set: { group: "30+" } }
)
```

8. DELETE - 데이터 삭제

```javascript
db.users.deleteOne({ username: "kim" })
db.users.deleteMany({ active: false })
```

주의: 다음 명령은 users의 모든 문서를 삭제함.

```javascript
db.users.deleteMany({})
```

다음 명령은 컬렉션 자체를 삭제함.

```javascript
db.users.drop()
```

9. 검색, 정렬, 개수, 인덱스

```javascript
db.users.find().sort({ age: 1 })
db.users.find().sort({ salary: -1 }).limit(3)
db.users.countDocuments()
db.users.countDocuments({ age: { $gte: 30 } })
db.users.createIndex({ username: 1 }, { unique: true })
db.users.getIndexes()
```

10. CRUD 핵심 암기표

구분	대표 명령	의미
CREATE	db.users.insertOne({...})	한 건 입력
CREATE	db.users.insertMany([...])	여러 건 입력
READ	db.users.find()	전체 조회
READ	db.users.findOne({name:"홍길동"})	한 건 조회
UPDATE	db.users.updateOne({...}, {$set:{...}})	한 건 수정
DELETE	db.users.deleteOne({...})	한 건 삭제

11. MongoDB 실무·학습용 명령어 100개

명령은 Docker 셸 명령과 mongosh 명령을 함께 정리함. 삭제 명령은 실습 DB에서만 주의해서 사용할 것.

번호	분류	목적	명령어
1	Docker	컨테이너 실행 확인	docker ps
2	Docker	모든 컨테이너 확인	docker ps -a
3	Docker	MongoDB 시작	docker start wiz-mongodb
4	Docker	MongoDB 중지	docker stop wiz-mongodb
5	Docker	MongoDB 재시작	docker restart wiz-mongodb
6	Docker	MongoDB 로그	docker logs wiz-mongodb
7	Docker	최근 로그 30줄	docker logs --tail 30 wiz-mongodb
8	Docker	Compose 시작	docker compose up -d
9	Docker	Compose 종료	docker compose down
10	접속	root 접속	docker exec -it wiz-mongodb mongosh -u root -p --authenticationDatabase admin
11	접속	devuser 접속	docker exec -it wiz-mongodb mongosh -u devuser -p --authenticationDatabase wizdb wizdb
12	DB	DB 목록	show dbs
13	DB	현재 DB	db
14	DB	wizdb 선택	use wizdb
15	DB	컬렉션 목록	show collections
16	DB	users 생성	db.createCollection("users")
17	DB	컬렉션 통계	db.users.stats()
18	DB	DB 통계	db.stats()
19	DB	users 컬렉션 삭제	db.users.drop()
20	DB	현재 DB 삭제	db.dropDatabase()
21	입력	한 건 입력	db.users.insertOne({name:"홍길동"})
22	입력	여러 건 입력	db.users.insertMany([{name:"김철수"},{name:"이영희"}])
23	입력	숫자 입력	db.users.insertOne({age:30})
24	입력	Boolean 입력	db.users.insertOne({active:true})
25	입력	날짜 입력	db.users.insertOne({reg_date:new Date()})
26	입력	배열 입력	db.users.insertOne({skills:["Java","Kotlin"]})
27	입력	중첩 객체 입력	db.users.insertOne({address:{city:"서울"}})
28	조회	전체 조회	db.users.find()
29	조회	한 건 조회	db.users.findOne()
30	조회	이름 검색	db.users.find({name:"홍길동"})
31	조회	username 검색	db.users.find({username:"hong"})
32	조회	나이 30	db.users.find({age:30})
33	조회	30세 이상	db.users.find({age:{$gte:30}})

번호	분류	목적	명령어
34	조회	30세 초과	db.users.find({age:{$gt:30}})
35	조회	30세 이하	db.users.find({age:{$lte:30}})
36	조회	30세 미만	db.users.find({age:{$lt:30}})
37	조회	30세 제외	db.users.find({age:{$ne:30}})
38	조회	25·30세	db.users.find({age:{$in:[25,30]}})
39	조회	25·30세 제외	db.users.find({age:{$nin:[25,30]}})
40	조회	AND 검색	db.users.find({age:30,gender:"남"})
41	조회	명시적 AND	db.users.find({$and:[{age:30},{gender:"남"}]})
42	조회	OR 검색	db.users.find({$or:[{age:25},{age:30}]})
43	조회	이름만 표시	db.users.find({}, {_id:0,name:1})
44	조회	이름·나이 표시	db.users.find({}, {_id:0,name:1,age:1})
45	조회	이름·직업 표시	db.users.find({}, {_id:0,name:1,job:1})
46	조회	_id 제외	db.users.find({}, {_id:0})
47	정렬	나이 오름차순	db.users.find().sort({age:1})
48	정렬	나이 내림차순	db.users.find().sort({age:-1})
49	정렬	급여 내림차순	db.users.find().sort({salary:-1})
50	조회	5건 제한	db.users.find().limit(5)
51	조회	5건 건너뛰기	db.users.find().skip(5)
52	정렬	급여 상위 3명	db.users.find().sort({salary:-1}).limit(3)
53	집계	전체 건수	db.users.countDocuments()
54	집계	남성 회원 수	db.users.countDocuments({gender:"남"})
55	집계	30세 이상 수	db.users.countDocuments({age:{$gte:30}})
56	수정	나이 변경	db.users.updateOne({username:"hong"},{$set:{age:31}})
57	수정	직업 변경	db.users.updateOne({username:"hong"},{$set:{job:"교수"}})
58	수정	급여 변경	db.users.updateOne({username:"hong"},{$set:{salary:5500000}})
59	수정	여러 필드 변경	db.users.updateOne({username:"hong"},{$set:{age:31,job:"교수"}})
60	수정	급여 증가	db.users.updateOne({username:"hong"},{$inc:{salary:100000}})
61	수정	나이 1 증가	db.users.updateOne({username:"hong"},{$inc:{age:1}})
62	수정	phone 필드 삭제	db.users.updateOne({username:"hong"},{$unset:{phone:""}})
63	수정	여러 문서 수정	db.users.updateMany({gender:"남"},{$set:{active:true}})
64	수정	조건부 여러 건	db.users.updateMany({age:{$gte:30}},{$set:{group:"30+"}})
65	수정	Upsert	db.users.updateOne({username:"new"},{$set:{name:"신규"}},{upsert:true})
66	삭제	한 건 삭제	db.users.deleteOne({username:"hong"})

번호	분류	목적	명령어
67	삭제	이름으로 삭제	db.users.deleteOne({name:"홍길동"})
68	삭제	비활성 회원 삭제	db.users.deleteMany({active:false})
69	삭제	20세 미만 삭제	db.users.deleteMany({age:{$lt:20}})
70	삭제	모든 문서 삭제	db.users.deleteMany({})
71	인덱스	username 인덱스	db.users.createIndex({username:1})
72	인덱스	username UNIQUE	db.users.createIndex({username:1},{unique:true})
73	인덱스	email UNIQUE	db.users.createIndex({email:1},{unique:true})
74	인덱스	복합 인덱스	db.users.createIndex({name:1,age:1})
75	인덱스	인덱스 목록	db.users.getIndexes()
76	인덱스	username 인덱스 삭제	db.users.dropIndex("username_1")
77	인덱스	보조 인덱스 전체 삭제	db.users.dropIndexes()
78	검색	이름 정규식	db.users.find({name:/김/})
79	검색	대소문자 무시	db.users.find({username:/hong/i})
80	검색	email 필드 존재	db.users.find({email:{$exists:true}})
81	배열	배열 값 검색	db.users.find({skills:"Kotlin"})
82	배열	배열 값 추가	db.users.updateOne({username:"hong"},{$push:{skills:"Rust"}})
83	배열	중복 없이 추가	db.users.updateOne({username:"hong"},{$addToSet:{skills:"Java"}})
84	배열	배열 값 제거	db.users.updateOne({username:"hong"},{$pull:{skills:"Rust"}})
85	집계	평균 급여	db.users.aggregate([{$group:{_id:null,avg:{$avg:"$salary"}}}])
86	집계	최고 급여	db.users.aggregate([{$group:{_id:null,max:{$max:"$salary"}}}])
87	집계	최저 급여	db.users.aggregate([{$group:{_id:null,min:{$min:"$salary"}}}])
88	집계	급여 합계	db.users.aggregate([{$group:{_id:null,total:{$sum:"$salary"}}}])
89	집계	직업별 인원	db.users.aggregate([{$group:{_id:"$job",count:{$sum:1}}}])
90	집계	성별 인원	db.users.aggregate([{$group:{_id:"$gender",count:{$sum:1}}}])
91	사용자	사용자 목록	show users
92	사용자	devuser 조회	db.getUser("devuser")
93	사용자	비밀번호 변경	db.changeUserPassword("devuser","새비밀번호")
94	사용자	devuser 삭제	db.dropUser("devuser")
95	관리	admin DB	use admin
96	관리	서버 상태	db.serverStatus()
97	관리	MongoDB 버전	db.version()
98	관리	현재 연결 정보	db.getMongo()
99	도움말	mongosh 도움말	help

번호	분류	목적	명령어
100	종료	mongosh 종료	exit

12. 초보자가 꼭 기억할 10개

```javascript
use wizdb
show dbs
show collections
db.createCollection("users")
db.users.insertOne({...})
db.users.insertMany([...])
db.users.find()
db.users.findOne({name:"홍길동"})
db.users.updateOne({name:"홍길동"}, {$set:{age:31}})
db.users.deleteOne({name:"홍길동"})
```

13. 실습 시 주의사항

- docker compose down -v 는 볼륨 삭제로 이어질 수 있으므로 데이터가 필요한 서버에서는 함부로 사용하지 않음.
- db.users.deleteMany({}) 는 컬렉션의 모든 문서를 삭제하므로 조건을 먼저 find()로 확인한 뒤 실행함.
- db.dropDatabase() 는 현재 선택된 데이터베이스 전체를 삭제하므로 실습 DB가 맞는지 db 명령으로 먼저 확인함.
- MongoDB 사용자는 생성된 인증 DB가 중요함. devuser이 wizdb에 생성되었다면 --authenticationDatabase wizdb 로 접속함.
- 외부 접속을 허용할 때는 27017 포트를 인터넷 전체에 무방비로 공개하지 말고 방화벽과 인증을 함께 적용함.

부록 · Docker 로 DB 설치

Docker Oracle 설치 교본

Rocky Linux + Docker Compose · Oracle AI Database Free · 초보자용

목표: Oracle 공식 Free 컨테이너 이미지를 사용하여 Docker Compose로 Oracle을 설치하고, 데이터 영구저장, 1521 외부접속, devuser 사용자/스키마 생성, DBeaver 연동까지 구성함.

1. 이 교본에서 사용하는 Oracle

Oracle 공식 컨테이너 이미지 container-registry.oracle.com/database/free:latest 를 사용함. 기본 컨테이너 데이터 경로는 /opt/oracle/oradata 이며, 기본 서비스는 FREEPDB1을 사용함.

Oracle AI Database Free는 학습·개발에 편리하지만 Free 에디션의 CPU·메모리·사용자 데이터 용량 제한 및 지원 정책이 있으므로 대규모 상용 운영은 별도 라이선스/지원 정책을 검토해야 함.

2. 최종 디렉터리 구조

```text
/home/devuser/docker/oracle/
├── compose.yml
├── .env
└── data/                ← 실제 Oracle DB 데이터 영구 저장
```

MariaDB/PostgreSQL과 마찬가지로 실제 DB 데이터는 ~/docker/oracle/data 에 두어 컨테이너와 분리함.

3. Docker 설치 확인

```bash
docker --version
docker compose version
sudo systemctl status docker
```

```bash
sudo systemctl enable --now docker
```

4. 기존 Oracle/1521 포트 확인

```bash
sudo ss -lntp | grep 1521
docker ps -a
```

1521을 이미 다른 Oracle이나 컨테이너가 사용 중이면 먼저 해당 서비스를 확인해야 함.

5. Oracle 작업 폴더 만들기

```bash
mkdir -p ~/docker/oracle/data
cd ~/docker/oracle
pwd
ls -la
```

6. .env 만들기

```bash
nano .env
```

```env
ORACLE_PASSWORD=본인이정한강력한비밀번호
ORACLE_PORT=1521
```

```bash
chmod 600 .env
```

ORACLE_PASSWORD는 SYSTEM/SYS 관리 계정 접속에 사용하는 비밀번호임. 충분히 강한 비밀번호를 사용함.

7. compose.yml 만들기

```bash
nano compose.yml
```

```yaml
services:
  oracle:
    image: container-registry.oracle.com/database/free:latest
    container_name: wiz-oracle
    restart: unless-stopped

    environment:
      ORACLE_PWD: ${ORACLE_PASSWORD}

    ports:
      - "${ORACLE_PORT}:1521"

    volumes:
      - ./data:/opt/oracle/oradata:Z

    healthcheck:
      test: ["CMD-SHELL", "healthcheck.sh"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 5m
```

핵심은 ./data:/opt/oracle/oradata:Z 임. Rocky Linux SELinux 환경을 고려해 :Z를 사용함. Oracle 초기화는 PostgreSQL/MariaDB보다 오래 걸릴 수 있음.

8. Compose 문법 검사

```bash
docker compose config
```

YAML 오류가 없을 때만 다음 단계로 진행함. TAB 대신 공백을 사용함.

9. Oracle 이미지 받기 및 시작

```bash
docker compose pull
docker compose up -d
docker ps
```

Oracle 공식 이미지는 크기가 크므로 최초 다운로드와 DB 초기화에 시간이 걸릴 수 있음.

10. Oracle 초기화 로그 확인

```bash
docker logs -f wiz-oracle
```

로그에서 DATABASE IS READY TO USE! 와 같은 준비 완료 메시지가 나타날 때까지 기다림. 로그 보기 종료는 Ctrl+C이며 컨테이너는 계속 실행됨.

11. 1521 포트 확인

```bash
sudo ss -lntp | grep 1521
```

정상이라면 0.0.0.0:1521 또는 [::]:1521에서 docker-proxy가 LISTEN 중임.

12. SYSTEM으로 Oracle 접속

```bash
docker exec -it wiz-oracle sqlplus system/"$ORACLE_PWD"@FREEPDB1
```

위 방식은 셸 환경에 ORACLE_PWD가 없을 수 있으므로 초보자는 다음처럼 컨테이너 내부 셸에 들어가 접속하는 방식도 편리함.

```bash
docker exec -it wiz-oracle bash
sqlplus system@FREEPDB1
```

비밀번호를 묻는 경우 .env에서 설정한 ORACLE_PASSWORD를 입력함.

```sql
SELECT sys_context('USERENV','CON_NAME') AS container_name FROM dual;
SHOW USER;
EXIT;
```

13. devuser 사용자(스키마) 생성

Oracle에서 USER는 로그인 계정이면서 기본적으로 같은 이름의 스키마와 연결됨. SYSTEM으로 FREEPDB1에 접속한 상태에서 실행함.

```sql
CREATE USER devuser IDENTIFIED BY "강력한비밀번호";

GRANT CREATE SESSION TO devuser;
GRANT CREATE TABLE TO devuser;
GRANT CREATE VIEW TO devuser;
GRANT CREATE SEQUENCE TO devuser;
GRANT CREATE PROCEDURE TO devuser;

ALTER USER devuser QUOTA UNLIMITED ON USERS;
```

학습용으로 필요한 일반 개발 권한을 부여한 예임. DBA 권한을 무조건 주는 것보다 필요한 권한만 부여하는 것이 안전함.

14. devuser에게 DBA 권한이 꼭 필요한 경우

```sql
GRANT DBA TO devuser;
```

DBA는 매우 강한 관리자 권한이므로 개인 학습 서버에서는 사용할 수 있지만 실제 서비스 애플리케이션 계정에는 권장하지 않음.

15. devuser으로 접속

```bash
docker exec -it wiz-oracle sqlplus devuser@FREEPDB1
```

비밀번호 입력 후 다음으로 확인함.

```sql
SHOW USER;
SELECT sys_context('USERENV','CON_NAME') FROM dual;
```

16. users 예제 테이블

```sql
CREATE TABLE users (
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    username VARCHAR2(50) NOT NULL UNIQUE,
    password VARCHAR2(255) NOT NULL,
    name VARCHAR2(100) NOT NULL,
    email VARCHAR2(150) UNIQUE,
    phone VARCHAR2(20),
    role VARCHAR2(20) DEFAULT 'USER',
    enabled NUMBER(1) DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
INSERT INTO users
(username, password, name, email, phone, role)
VALUES
('hong', '1234', '홍길동', 'hong@test.com', '010-3333-4444', 'USER');

COMMIT;
```

실제 로그인 서비스에서는 password를 평문으로 저장하지 않고 BCrypt 등으로 해시하여 저장해야 함.

17. 테이블 조회

```sql
SELECT table_name FROM user_tables;
DESC users;
SELECT * FROM users;
```

SQL*Plus에서 긴 결과가 불편하면 LINESIZE와 PAGESIZE를 조정할 수 있음.

```sql
SET LINESIZE 200
SET PAGESIZE 100
SELECT * FROM users;
```

18. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 Oracle 1521을 허용하는 구성을 권장함.

```bash
sudo firewall-cmd --permanent --remove-port=1521/tcp

sudo firewall-cmd --permanent   --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="1521" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

기존에 1521/tcp 전체 허용이 없으면 remove-port 명령이 실패하거나 변경 없음으로 나올 수 있으며, 그 경우 rich rule 추가 여부를 확인하면 됨.

19. Windows에서 1521 테스트

현재 Rocky Linux 서버 내부 IP SERVER_IP 기준:

```powershell
Test-NetConnection SERVER_IP -Port 1521
```

TcpTestSucceeded : True이면 Windows에서 Docker Oracle listener까지 연결됨.

20. DBeaver 연결

항목	값
DB 종류	Oracle
Host	SERVER_IP
Port	1521
접속 방식	Service Name
Service Name	FREEPDB1
Username	devuser
Password	devuser 생성 시 설정한 비밀번호

```text
jdbc:oracle:thin:@//SERVER_IP:1521/FREEPDB1
```

SID가 아니라 Service Name=FREEPDB1을 사용하는 것이 핵심임.

21. 데이터 영구저장 원리

```text
Rocky Linux
/home/devuser/docker/oracle/data
              |
              | Bind Mount
              ▼
Oracle Container
/opt/oracle/oradata
```

> `docker compose down`으로 컨테이너를 제거해도 `data` 폴더는 남음. 같은 `compose.yml`로 다시 올리면 기존 데이터 파 bash
> 일을 사용함.

주의:

> `~/docker/oracle/data`를 삭제하면 실제 `Oracle` 데이터 파일을 잃을 수 있으므로 함부로 삭제하지 않음. text

22. 자주 쓰는 Docker 명령

목적	명령
설정 검사	docker compose config
이미지 받기	docker compose pull
시작	docker compose up -d
상태	docker ps
로그	docker logs wiz-oracle
실시간 로그	docker logs -f wiz-oracle
재시작	docker compose restart
중지	docker compose stop
컨테이너 제거	docker compose down
SYSTEM 접속	docker exec -it wiz-oracle sqlplus system@FREEPDB1
devuser 접속	docker exec -it wiz-oracle sqlplus devuser@FREEPDB1

23. 백업 기초

Oracle은 Data Pump(expdp/impdp), RMAN 등 여러 백업 방식이 있음. 초보자 단계에서는 DB 데이터 디렉터리를 단순 복사하는 것만으로 운영 중인 Oracle의 일관된 백업을 보장한다고 생각하면 안 됨. 실제 운영 백업은 RMAN 또는 Data Pump를 학습하여 구성하는 것을 권장함.

24. MariaDB/PostgreSQL과 다른 점

항목	MariaDB/PostgreSQL	Oracle Free
기본 포트	3306 / 5432	1521
DB 접속 단위	Database	PDB Service
기본 서비스	wizdb 등	FREEPDB1
사용자	계정 중심	사용자와 스키마가 밀접
데이터 경로	/var/lib/...	/opt/oracle/oradata
초기 시작	비교적 빠름	상대적으로 오래 걸림
관리 CLI	mariadb / psql	sqlplus

25. 초보자가 기억할 핵심 명령

```bash
cd ~/docker/oracle
docker compose config
docker compose up -d
docker logs -f wiz-oracle
sudo ss -lntp | grep 1521
docker exec -it wiz-oracle sqlplus system@FREEPDB1
docker exec -it wiz-oracle sqlplus devuser@FREEPDB1
docker compose down
```

26. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-oracle 실행 중
초기화	DB 준비 완료 로그 확인
영구저장	~/docker/oracle/data 사용
Listener	1521 LISTEN
PDB	FREEPDB1
사용자	devuser 접속 가능
방화벽	192.168.0.0/24 → 1521 허용
Windows 테스트	TcpTestSucceeded : True
DBeaver	Service Name FREEPDB1 연결

27. 참고: Oracle Free의 성격

Oracle 공식 Free 컨테이너 이미지는 학습·개발·프로토타이핑에 매우 편리함. Free 에디션은 리소스 제한이 있으며 별도 지원/패치 정책도 Enterprise/상용 지원 제품과 다르므로 중요 상용 시스템에서는 라이선스와 지원 요건을 별도로 검토함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.